

IEE3584 - Comunicaciones inalámbricas

Tarea 11

Nicolás Seguel

Noviembre 2025

Problema 28

(a) Generación de secuencias de desvanecimiento

Se generaron $N_r = 4$ secuencias de desvanecimiento Rayleigh utilizando la rutina `fading_rayleigh_fd()` del Problema 8, con los mismos parámetros:

$$v = 60 \text{ km/h}, \quad f_c = 1,8 \text{ GHz}, \quad T_s = 100 \text{ } \mu\text{s}, \quad T_d = 0,5 \text{ s}.$$

Cada secuencia usa una semilla distinta: [10, 20, 30, 40], y se normaliza para que $E[|h|^2] = 1$, lo que implica una razón señal a ruido media $\bar{\gamma} = 1$.

En la Figura 1 se muestran las envolventes instantáneas $|h_r(t)|$ de las cuatro ramas receptoras, superpuestas. Se verifica que el valor RMS de cada secuencia es cercano a 1 (0 dB).

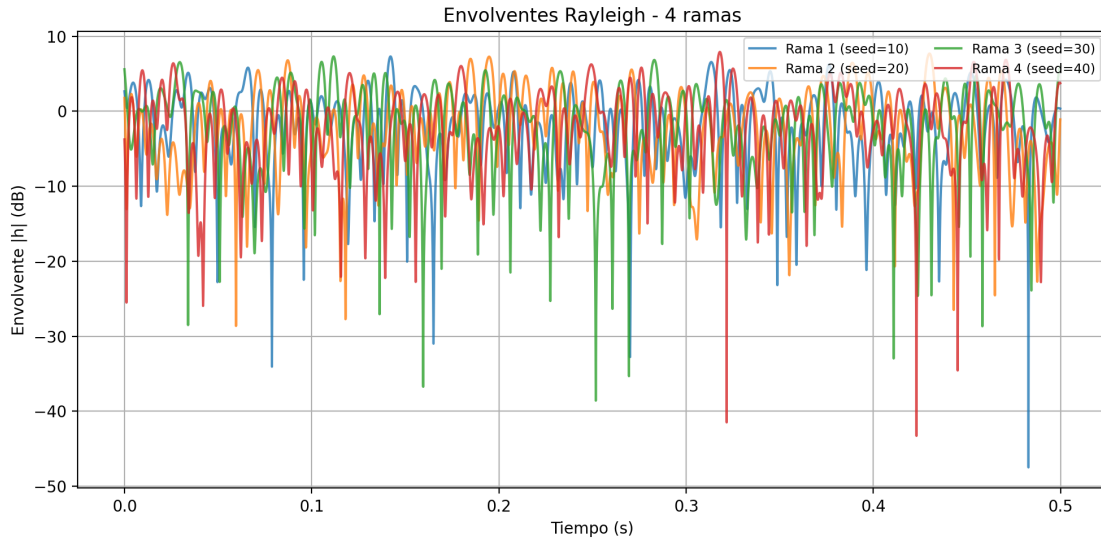


Figura 1: Envloventes Rayleigh $|h_r(t)|$ de las $N_r = 4$ ramas.

(b) Combinación MRC y ganancia de arreglo

Para cada instante discreto n , se calcularon los pesos MRC instantáneos:

$$\hat{u}_r^*(n) = \frac{h_r^*(n)}{\sqrt{\sum_{k=1}^{N_r} |h_k(n)|^2}},$$

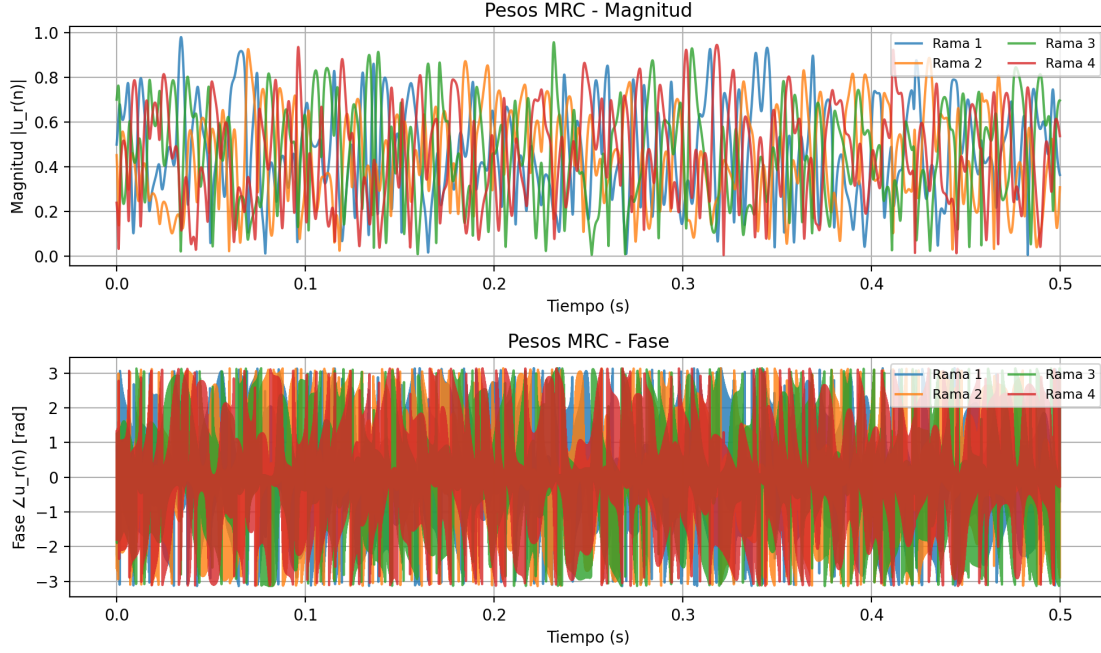


Figura 2: Pesos MRC $\hat{u}_r^*(n)$ para las $N_r = 4$ ramas.

de modo que el vector de pesos maximiza la SNR de salida.

La SNR instantánea de cada rama y de la salida se obtuvo como:

$$\gamma_r(n) = \bar{\gamma} |h_r(n)|^2, \quad \Gamma(n) = \bar{\gamma} \sum_{r=1}^{N_r} |h_r(n)|^2.$$

La Figura 3 muestra las SNR por rama, mientras que la Figura 4 presenta la SNR de salida del combinador.

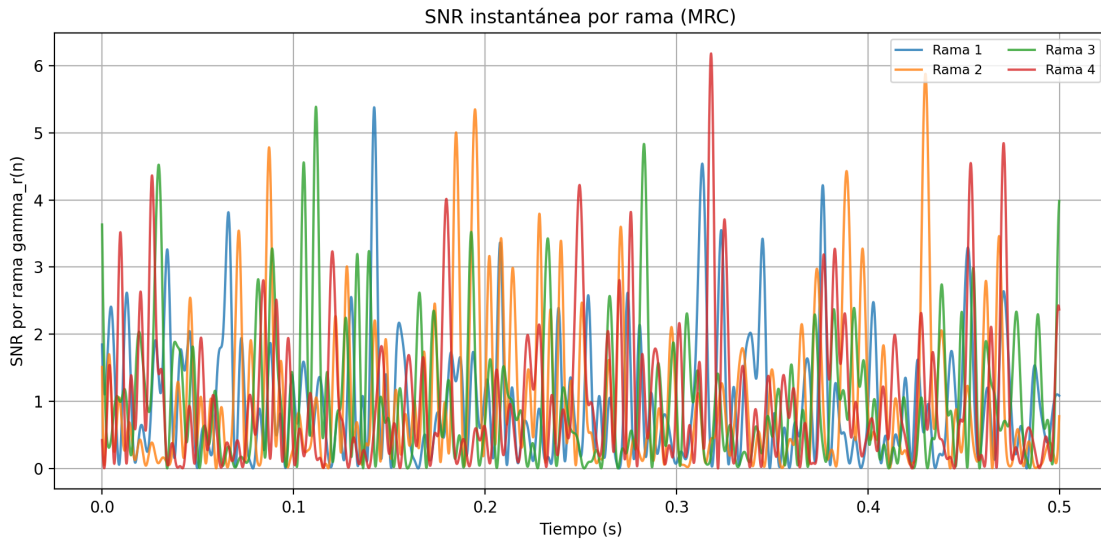


Figura 3: SNR instantánea por rama $\gamma_r(n)$ en valores lineales.

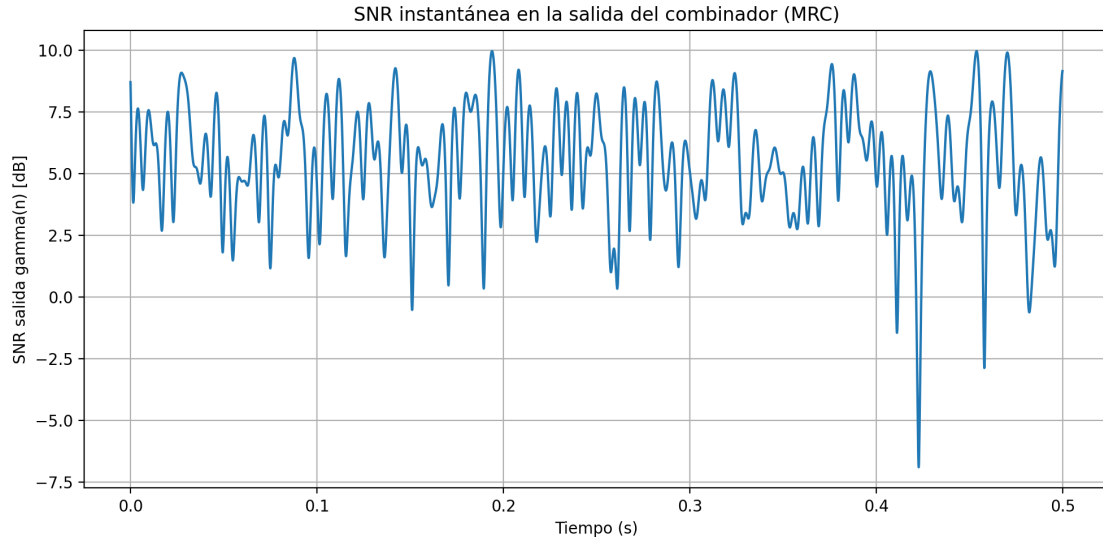


Figura 4: SNR instantánea a la salida del combinador MRC, en dB.

El promedio temporal de $\Gamma(n)$ entrega la SNR promedio de salida:

$$\bar{\Gamma} = E[\Gamma(n)] \approx 4,01,$$

de donde la ganancia de arreglo resulta:

$$G_{\text{arr}} = \frac{\bar{\Gamma}}{\bar{\gamma}} = 4,01 \approx 6,03 \text{ dB}.$$

Comentario

El resultado coincide con la predicción teórica de una ganancia de $N_r = 4$, es decir,

$$G_{\text{teo}} = 10 \log_{10}(N_r) = 10 \log_{10}(4) = 6,02 \text{ dB}.$$

Por tanto, el simulador confirma que el combinador MRC aprovecha completamente la diversidad espacial del arreglo receptor, aumentando linealmente la SNR media con el número de ramas.

Código fuente

```
#!/usr/bin/env python3
# problema28.py
# Genera Nr secuencias de desvanecimiento (usar la rutina del Problema 8)
# Calcula pesos MRC instantáneos, SNR por rama y SNR de salida, y grafica resultados.
# Requiere: numpy, matplotlib y la implementacion de fading_rayleigh_fd() del Problema 8.

import numpy as np
import matplotlib.pyplot as plt
from numpy import pi

# =====
# 1. FUNCIONES BASE (del Problema 8)
# =====

def clarke_psd(f, fD, eps=1e-9):
    """Densidad espectral de potencia (PSD) del modelo de Clarke."""
```

```

S = np.zeros_like(f, dtype=float)
mask = np.abs(f) < fD
x = f[mask] / (fD + 0.0)
denom = np.sqrt(np.maximum(1.0 - x**2, eps))
S[mask] = 1.0 / (pi * fD * denom)
return S

def fading_rayleigh_fd(v, fc, Ts, Td, seed=0, Nfreq=None):
    """Genera una secuencia Rayleigh correlacionada en el tiempo usando el método FD."""
    c = 3e8
    fD = (v * fc) / c # Doppler máximo
    N = int(np.round(Td / Ts))
    if Nfreq is None:
        Nfft = 1 << (int(np.ceil(np.log2(N))) + 2)
    else:
        Nfft = int(Nfreq)
    freqs = np.fft.fftfreq(Nfft, d=Ts)
    S = clarke_psd(freqs, fD)
    rng = np.random.default_rng(seed)
    spec = (rng.normal(size=Nfft) + 1j * rng.normal(size=Nfft)) * np.sqrt(S / 2.0)
    h_time = np.fft.ifft(np.fft.ifftshift(spec))
    h = h_time[:N]
    h /= np.sqrt(np.mean(np.abs(h)**2)) # normalización RMS = 1
    t = np.arange(N) * Ts
    return h, fD, t

def compute_mean_rms(r):
    """Calcula media y RMS de un vector real."""
    mean_r = np.mean(r)
    rms = np.sqrt(np.mean(r**2))
    return mean_r, rms

# =====
# 2. FUNCIONES DE MÓDULO RECEPTOR
# =====

def plot_envelopes(h_list, t, seeds, savepath="figs/problema28_envelopes.png"):
    """Grafica las envolventes de todas las ramas en dB."""
    Nr = h_list.shape[0]
    plt.figure(figsize=(10,5))
    for r in range(Nr):
        plt.plot(t, 20*np.log10(np.abs(h_list[r,:]) + 1e-12), label=f'Rama {r+1} (seed={seeds[r]}',
            ↪ alpha=0.8)
    plt.xlabel("Tiempo (s)")
    plt.ylabel("Envolvente |h| (dB)")
    plt.title(f"Envolventes Rayleigh - {Nr} ramas")
    plt.grid(True)
    plt.legend(loc='upper right', ncol=2, fontsize='small')
    plt.tight_layout()
    plt.savefig(savepath, dpi=200)
    plt.close()
    print(f"[OK] Figura guardada: {savepath}")

def mrc_weights(h_list):
    """
    Calcula los pesos MRC instantáneos:
    
$$u_r(n) = h_r(n) / \sqrt{\sum_k |h_k(n)|^2}$$

    """
    Nr, N = h_list.shape
    denom = np.sqrt(np.sum(np.abs(h_list)**2, axis=0) + 1e-18)
    weights = np.conj(h_list) / denom
    return weights

def plot_weights(weights, t, savepath="figs/problema28_mrc_weights.png"):
    """Grafica los pesos MRC instantáneos en magnitud y fase."""
    Nr = weights.shape[0]
    plt.figure(figsize=(10,6))

```

```

# Magnitud
plt.subplot(2,1,1)
for r in range(Nr):
    plt.plot(t, np.abs(weights[r,:]), label=f'Rama {r+1}', alpha=0.8)
plt.xlabel("Tiempo (s)")
plt.ylabel("Magnitud |u_r(n)|")
plt.title("Pesos MRC - Magnitud")
plt.grid(True)
plt.legend(loc='upper right', ncol=2, fontsize='small')

# Fase
plt.subplot(2,1,2)
for r in range(Nr):
    plt.plot(t, np.angle(weights[r,:]), label=f'Rama {r+1}', alpha=0.8)
plt.xlabel("Tiempo (s)")
plt.ylabel("Fase <u_r(n) [rad]")
plt.title("Pesos MRC - Fase")
plt.grid(True)
plt.legend(loc='upper right', ncol=2, fontsize='small')

plt.tight_layout()
plt.savefig(savepath, dpi=200)
plt.close()
print(f"[OK] Figura guardada: {savepath}")

def compute_snr_values(h_list, gamma_bar=1.0):
    """
    Calcula SNR por rama y total:
        gamma_r(n) = gamma_bar * |h_r(n)|^2
        gamma(n)   = gamma_bar * sum_r |h_r(n)|^2
    """
    gamma_r = gamma_bar * np.abs(h_list)**2
    Gamma_out = np.sum(gamma_r, axis=0)
    return gamma_r, Gamma_out

def plot_snr_per_branch(gamma_r, t, savepath="figs/problema28_gamma_r.png"):
    """Grafica la SNR por rama lineal."""
    Nr = gamma_r.shape[0]
    plt.figure(figsize=(10,5))
    for r in range(Nr):
        plt.plot(t, gamma_r[r,:], label=f'Rama {r+1}', alpha=0.8)
    plt.xlabel("Tiempo (s)")
    plt.ylabel("SNR por rama gamma_r(n)")
    plt.title("SNR instantánea por rama (MRC)")
    plt.grid(True)
    plt.legend(loc='upper right', ncol=2, fontsize='small')
    plt.tight_layout()
    plt.savefig(savepath, dpi=200)
    plt.close()
    print(f"[OK] Figura guardada: {savepath}")

def plot_snr_output(Gamma_out, t, savepath="figs/problema28_Gamma_out.png"):
    """Grafica la SNR instantánea de salida del combinador."""
    plt.figure(figsize=(10,5))
    plt.plot(t, 10*np.log10(Gamma_out + 1e-12))
    plt.xlabel("Tiempo (s)")
    plt.ylabel("SNR salida gamma(n) [dB]")
    plt.title("SNR instantánea en la salida del combinador (MRC)")
    plt.grid(True)
    plt.tight_layout()
    plt.savefig(savepath, dpi=200)
    plt.close()
    print(f"[OK] Figura guardada: {savepath}")

# =====
# 3. FUNCIÓN PRINCIPAL
# =====

```

```

def problema28_main():
    Nr = 4
    v = 60.0 / 3.6      # velocidad [m/s]
    fc = 1.8e9          # frecuencia portadora [Hz]
    Ts = 100e-6         # periodo de muestreo [s]
    Td = 0.5            # duración total [s]
    seeds = [10, 20, 30, 40]
    gamma_bar = 1.0     # SNR media por rama (lineal)

    # --- Generación de secuencias ---
    h_list = []
    for i in range(Nr):
        h, fD, t = fading_rayleigh_fd(v, fc, Ts, Td, seed=seeds[i])
        r = np.abs(h)
        mean_r, rms = compute_mean_rms(r)
        print(f"Rama {i+1}: Mean |h| = {mean_r:.4f}, RMS |h| = {rms:.4f}")
        h_list.append(h)
    h_list = np.vstack(h_list)

    # --- Gráficos de las envolventes ---
    plot_envelopes(h_list, t, seeds)

    # --- Pesos MRC y SNRs ---
    weights = mrc_weights(h_list)
    gamma_r, Gamma_out = compute_snr_values(h_list, gamma_bar)

    # --- Gráficos de pesos MRC ---
    plot_weights(weights, t)

    # --- Promedios y ganancia de arreglo ---
    Gamma_mean = np.mean(Gamma_out)
    array_gain = Gamma_mean / gamma_bar
    print(f"\nPromedio gamma_mean = {Gamma_mean:.4f} (lineal)")
    print(f"Ganancia de arreglo = {array_gain:.4f} = {10*np.log10(array_gain):.2f} dB")
    print(f"Teoría: G_arr = N_r = {Nr}, en dB = {10*np.log10(Nr):.2f} dB")

    # --- Graficar SNRs ---
    plot_snr_per_branch(gamma_r, t)
    plot_snr_output(Gamma_out, t)

    print("\nSimulación completada. Resultados guardados en carpeta figs/")

    # =====
    # 4. EJECUCIÓN DIRECTA
    # =====

    if __name__ == "__main__":
        problema28_main()

```